Systemadministration

Linux

Web

Agenda

1. The Web
2. HTTP
3. Website
4. Web Server
5. Proxying & Load Balancing
6. VRRP
7. Redundant arkitektur

The Web

Webben

Är Internet och webben samma sak?

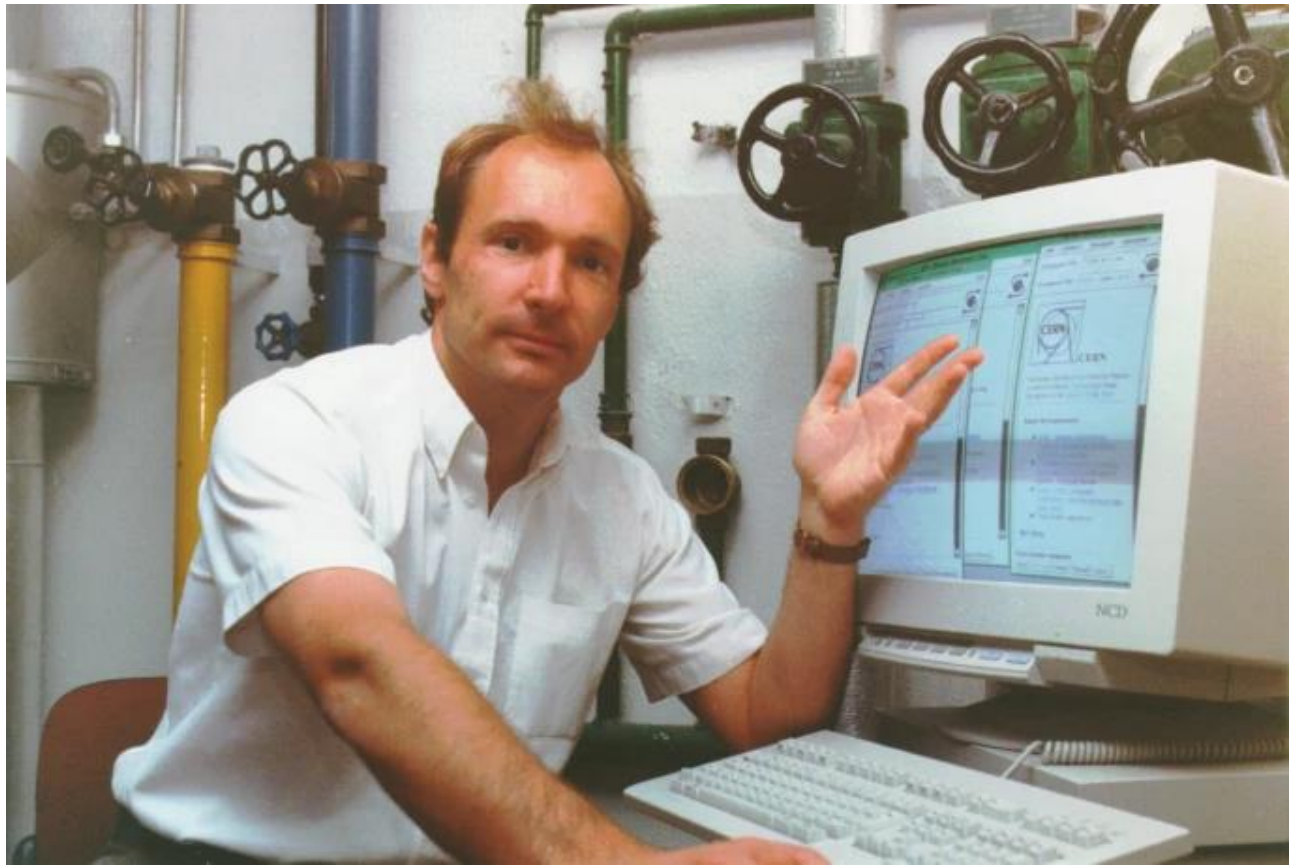
WWW's "historical" logo, created by Robert Cailliau in 1990. Made of three W using the Optima Bold font, according to Cailliau himself, by Fakefunk, 2022, Wikimedia Commons (https://commons.wikimedia.org/wiki/File:WWW-LetShare.svg). CC BY-SA 4.0 International

Tim Berners-Lee

Föreslog **WWW**

Utvecklade med Robert Cailliau:

- **HTML**
- **HTTP**
- **Webbläsare**



Tim Berners-Lee

“**Did you invent the Internet?**

No, no, no!

When I was doing the WWW, most of the bits I needed were already done [...]. People had already used TCP/IP and DNS to make email, and other cool things [...].

I didn't invent the hypertext link either. The idea of jumping from one document to another had been thought about lots of people [...]. Lots of hypertext systems had been made which just worked on one computer, and didn't link all the way across the world.

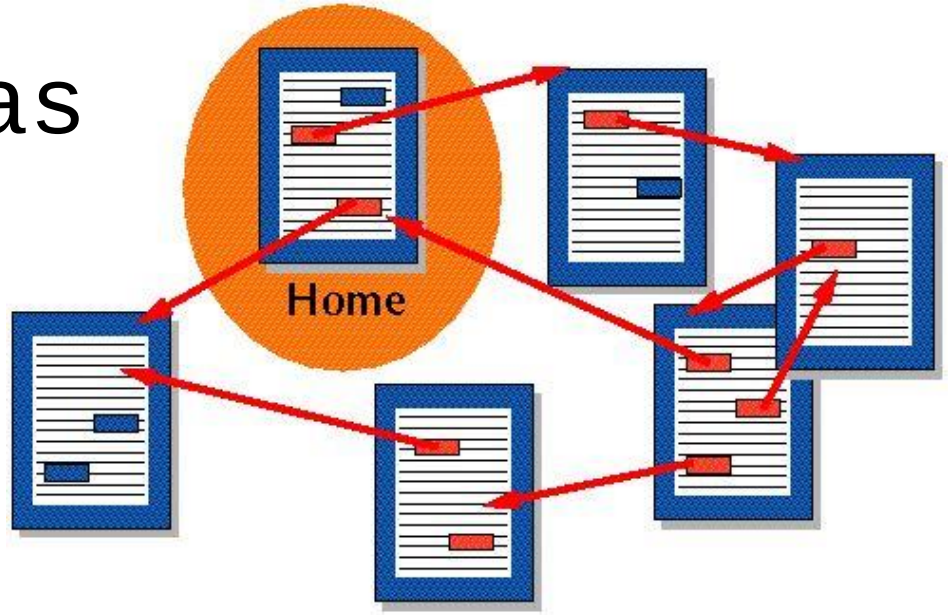
I just had to take the hypertext idea and connect it to the TCP and DNS ideas and -- ta-da! -- the World Wide Web.”

https://www.w3.org/People/Berners-Lee/Kids.html

HTTP

Hypertext

Dokument som sammankopplas med **hyperlinks**



Several documents being connected by hyperlinks, by Robert Cailliau, 1992, Wikimedia Commons (https://commons.wikimedia.org/wiki/File:Sistema_hipertextual.jpg). CC BY-SA 3.0 Unported

Webben använder:
HTML (**H**yper**T**ext **M**arkup **L**anguage)

Uniform **R**esource **L**ocator

URL: adresser i **webben**

Ex.

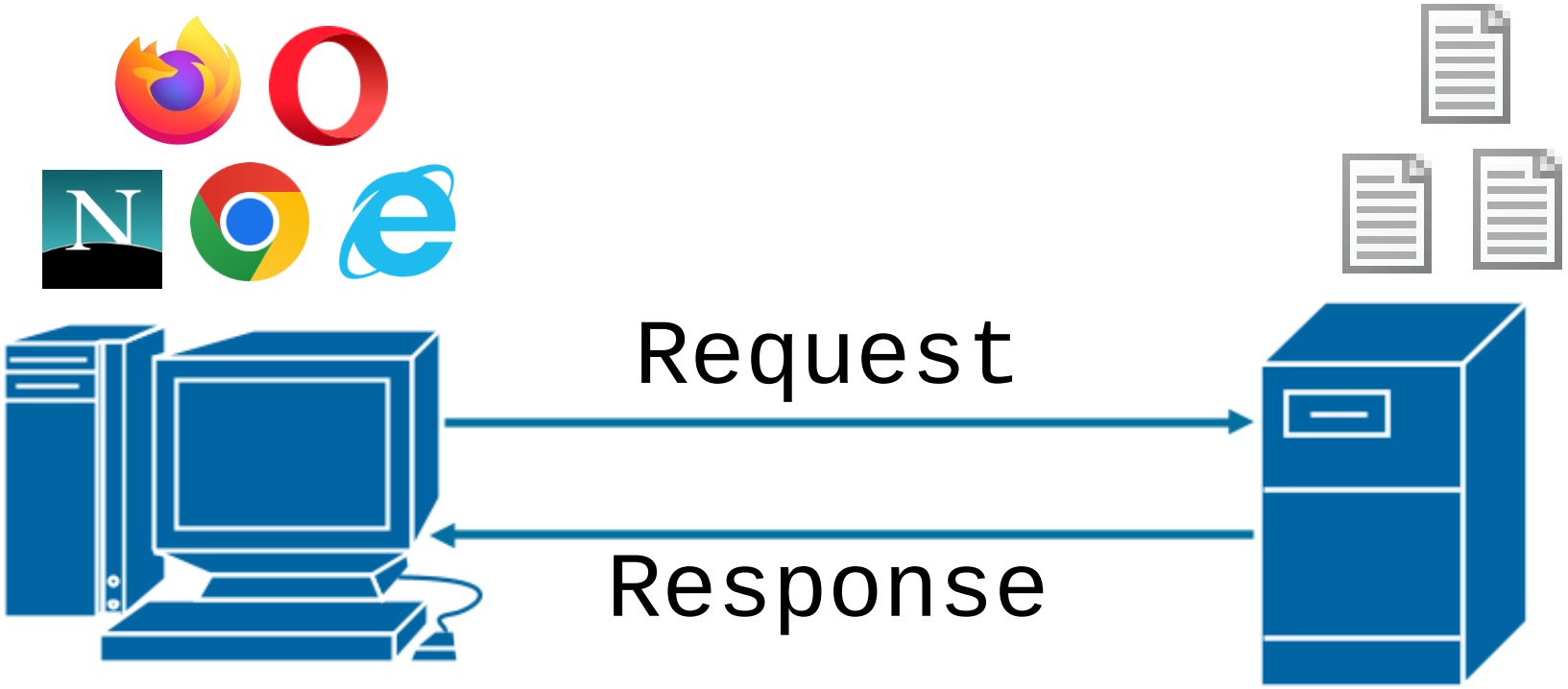
protocol://address:port/file-path

http://www.ju.se:80/index.html

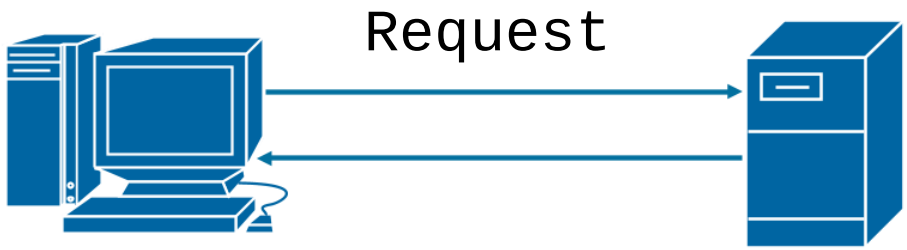
ju.se

Hyper**T**ext **T**ransport **P**rotocol

HTTP: i sin enklaste form ett protokoll för att hämta **HTML**-dokument



HTTP Request Methods



GET	Retrieves the resource
HEAD	Retrieves only the metadata about the resource
POST	Sends data to the resource
PUT	Creates, or replaces, the resource with the supplied data
DELETE	Removes the resource
OPTIONS	Returns the permitted request methods for the resource

HTTP Response Status Codes

1xx	Informational
2xx	Successful
3xx	Redirection
4xx	Client error
5xx	Server error

Vanliga:

200 OK

404 Not Found

curl

CLI-baserad webbläsare för Linux & Windows

GET URL

```
nics@linux:~$ curl https://ju.se
```

HEAD URL

```
nics@linux:~$ curl -I https://ju.se
```

Save HTML-file

```
nics@linux:~$ curl -O https://ju.se
```

Livedemo

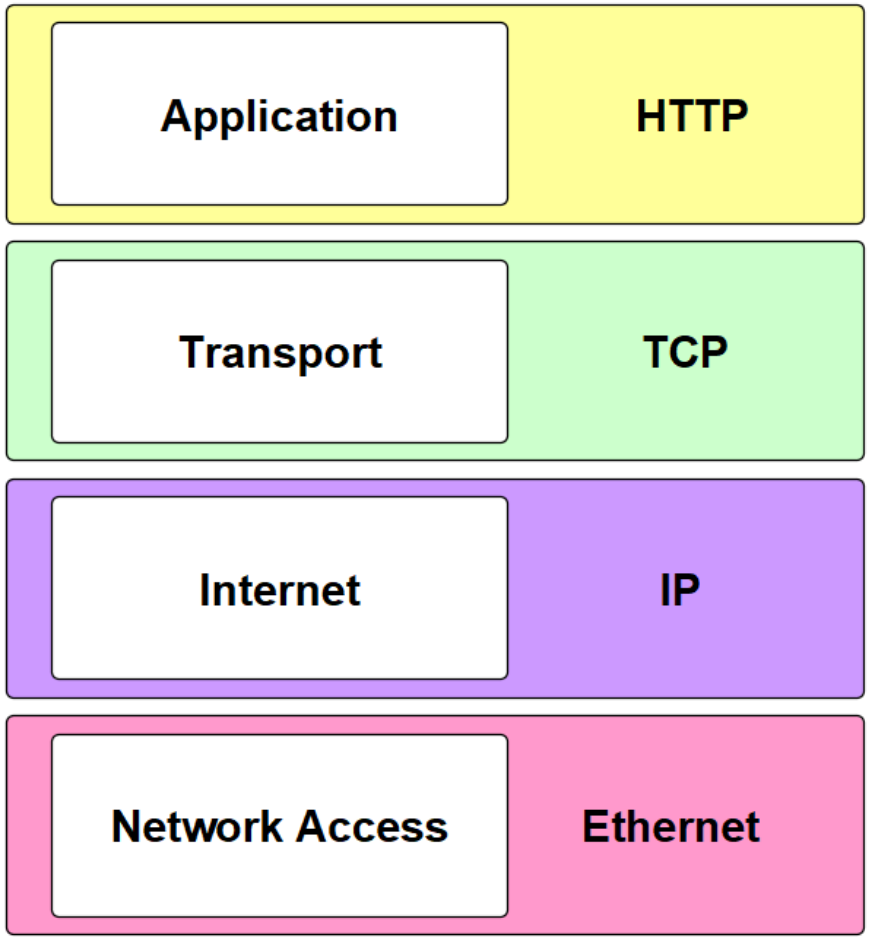
curl

Hyper**T**ext **T**ransport **P**rotocol

HTTP:

- **TCP** port **80**
- **Orkrypterat**

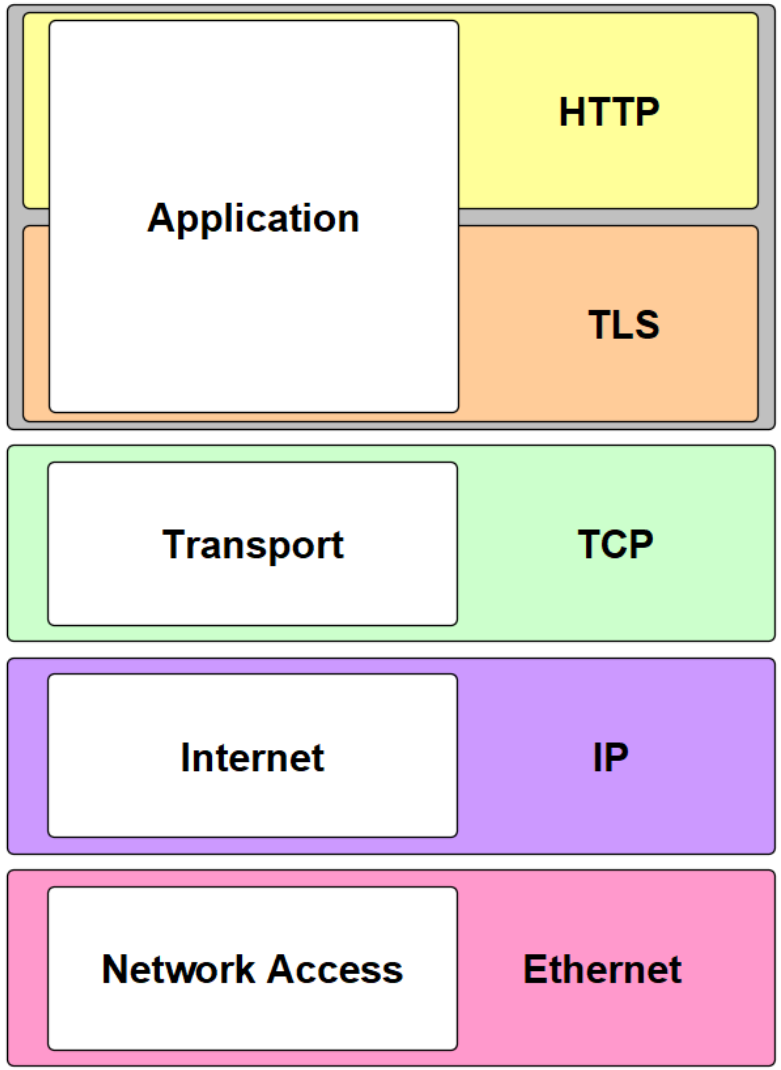
Beroende av **DNS**



Transport Layer Security

HTTPS:

- **HTTP** over **TLS**
- **TCP** port **443**
- Kryptering & server autentisering



X.509 Certificates

Innehåller bland annat:

Validity

Not Before: Nov 21 08:00:00 2016 UTC

Not After : Nov 22 07:59:59 2017 UTC

Subject Public Key Info:

Public-Key: (256 bit)

pub:

00:c9:22:69:31:8a:d6:6c:ea:da:c3:7f:2c…

Signature Algorithm: sha256WithRSAEncryption

8b:c3:ed:d1:9d:39:6f:af:40:72:bd:1e:18:5e…

X.509 Certificates

Validity

Hur länge certifikatet är giltigt

Identity (public key)

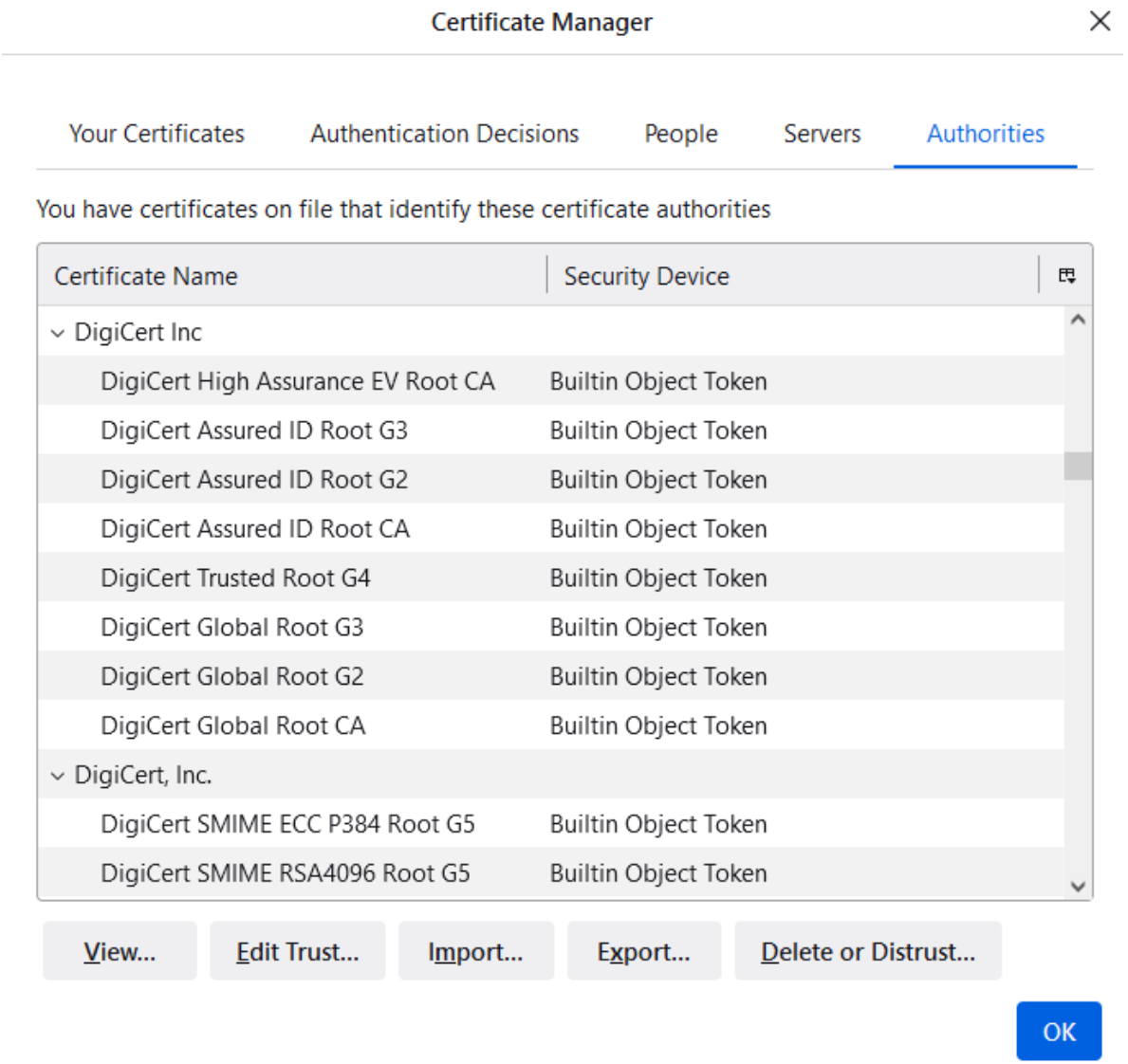
Den som autentiseras

Signature

Den som utfärdat certifikatet
och garanterar dess giltighet

X.509 Certificates Root of Trust

Webbläsare har en lista över vilka signaturer den litar på



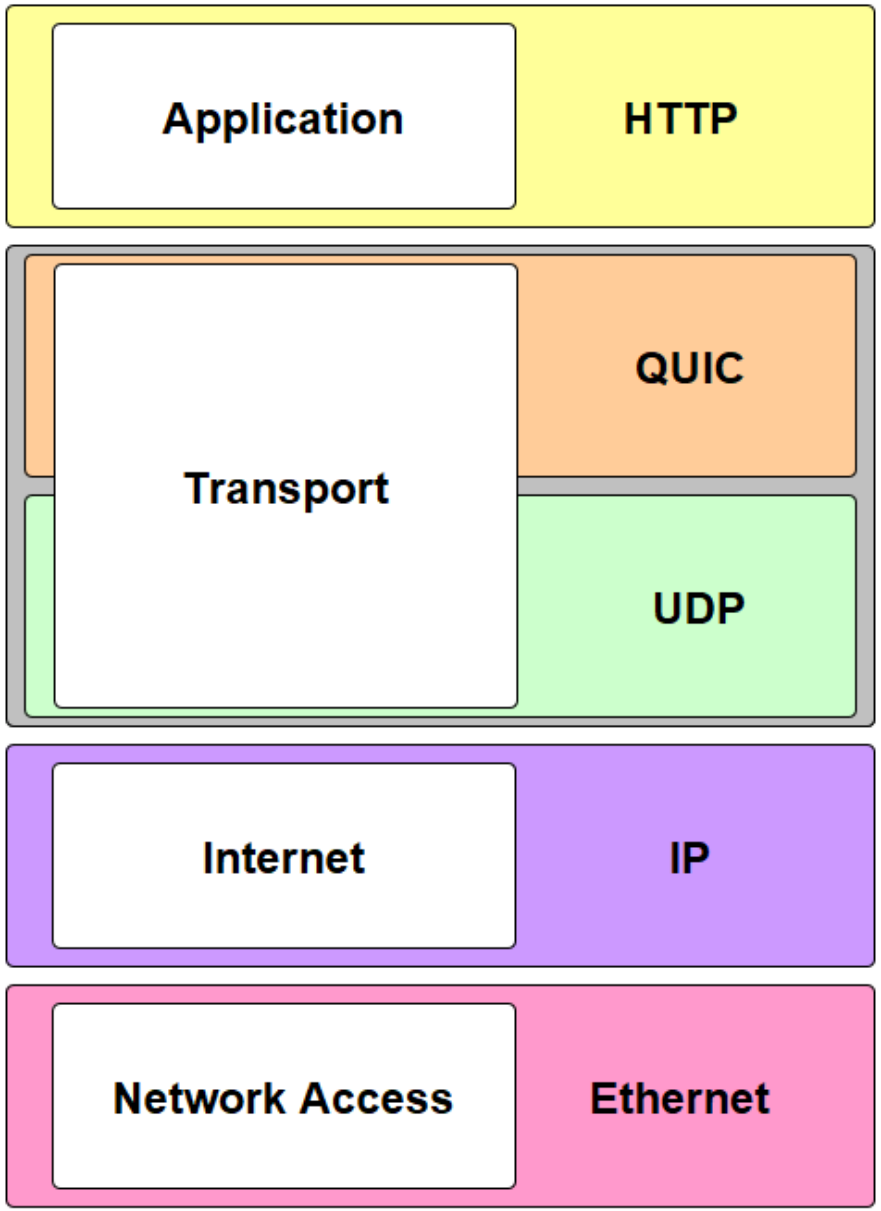
Livedemo

https://ju.se

QUIC

Standard: **HTTP/3** 2022

- **UDP** port **443**
- Alltid **TLS** kryptering & autentisering
- Kortare round-trip
- **QUIC** hanterar ’TCP-funktioner’



Website

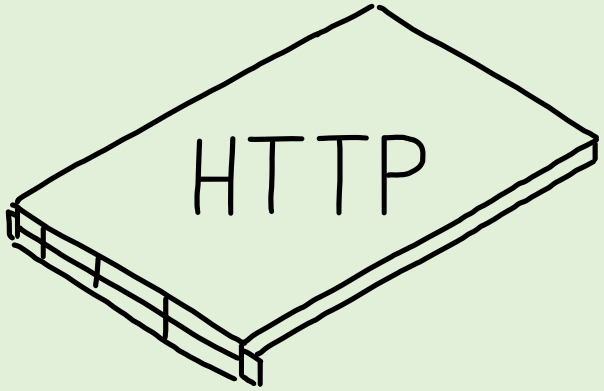
Web Page
Webbsida

ju.se/nics
ju.se/nmd
ju.se/dis
ju.se/dmp

Web Site
Webbplats

hj.se ju.se

Web Server
Webbserver



193.10.208.54

Språk i webben

Moderna **webbplatser/websites** använder:

- **HTML** Content
- **CSS** Presentation
- **Javascript** Logic

Webpages

Webbsidor

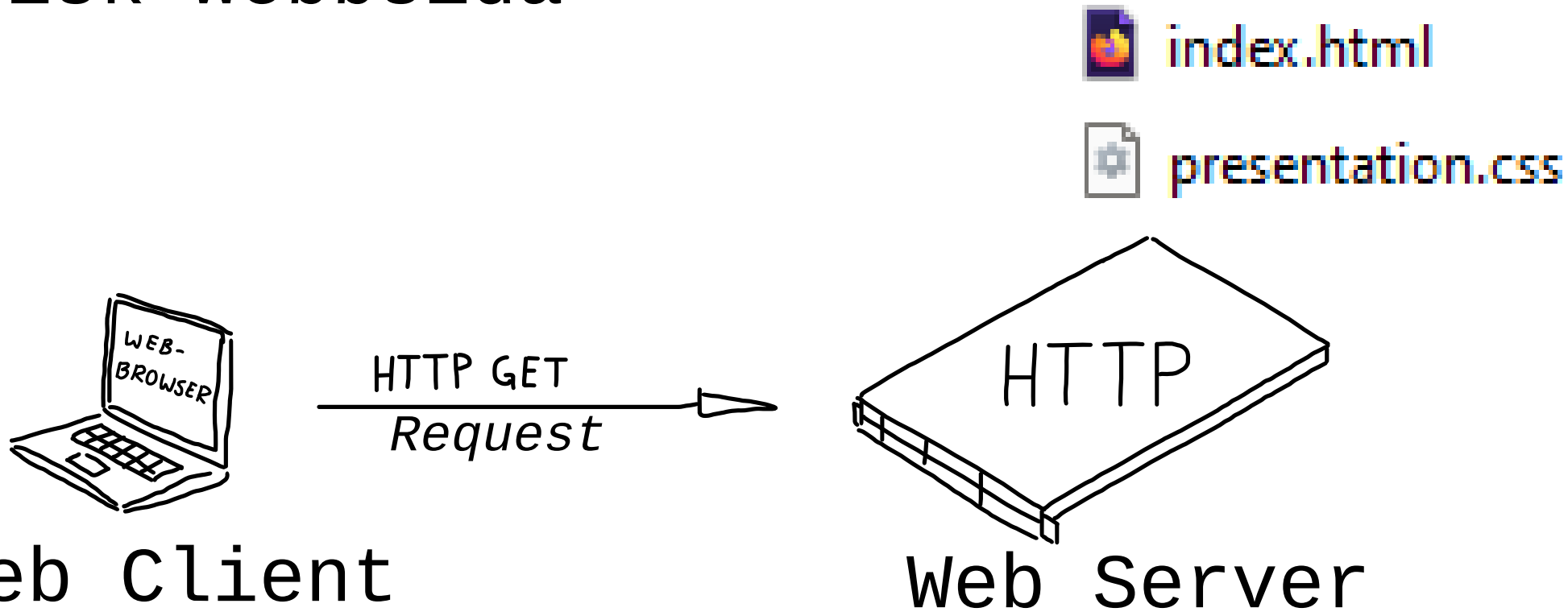
webbserver ≈ filserver

Webbsidor är antingen:

- **Static** (HTML+CSS)
- **Dynamic** (Javascript etc.)
  - Client-side-scripting
  - Server-side-scripting

Static Webpage

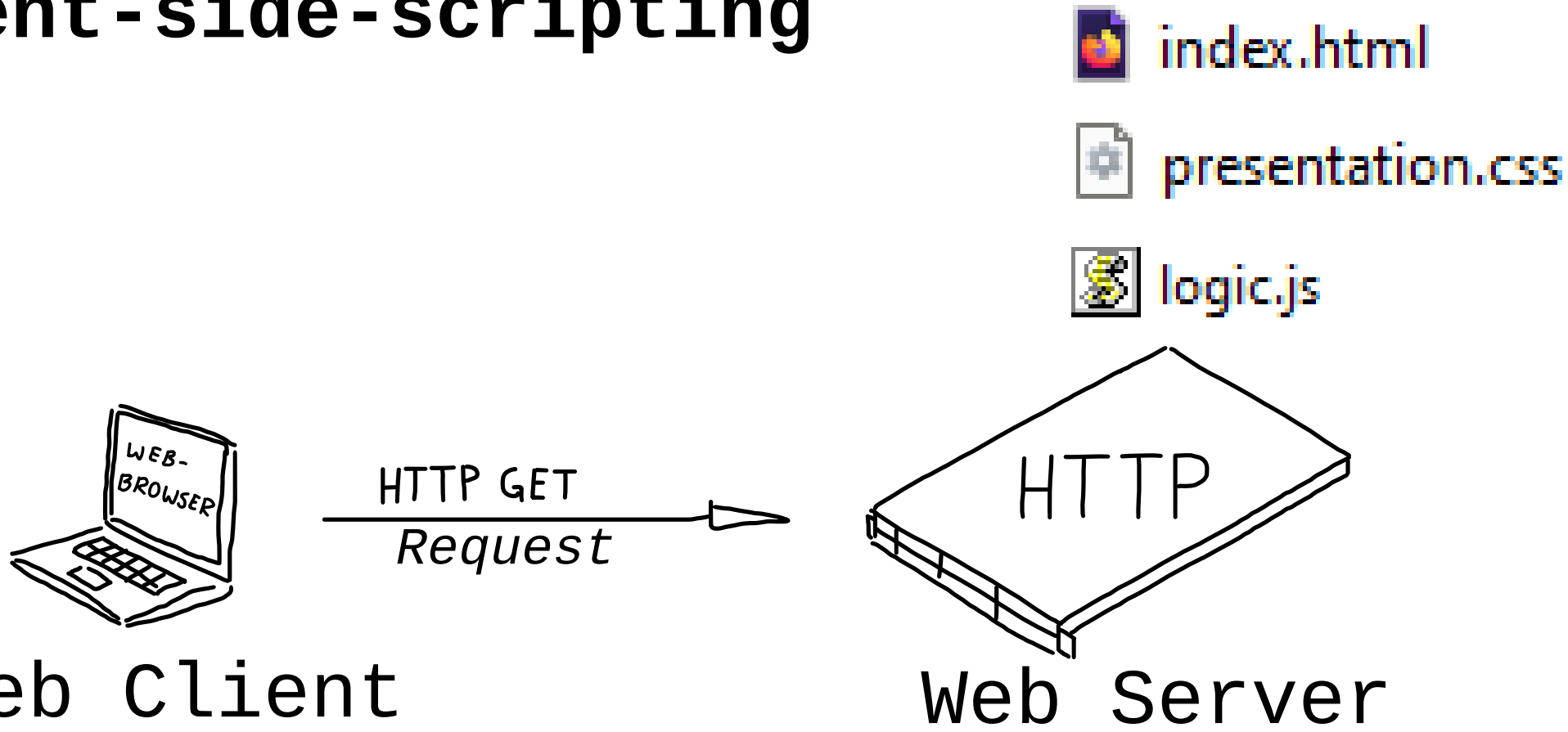
Statisk webbsida



Dynamic Webpage

Dynamisk webbsida

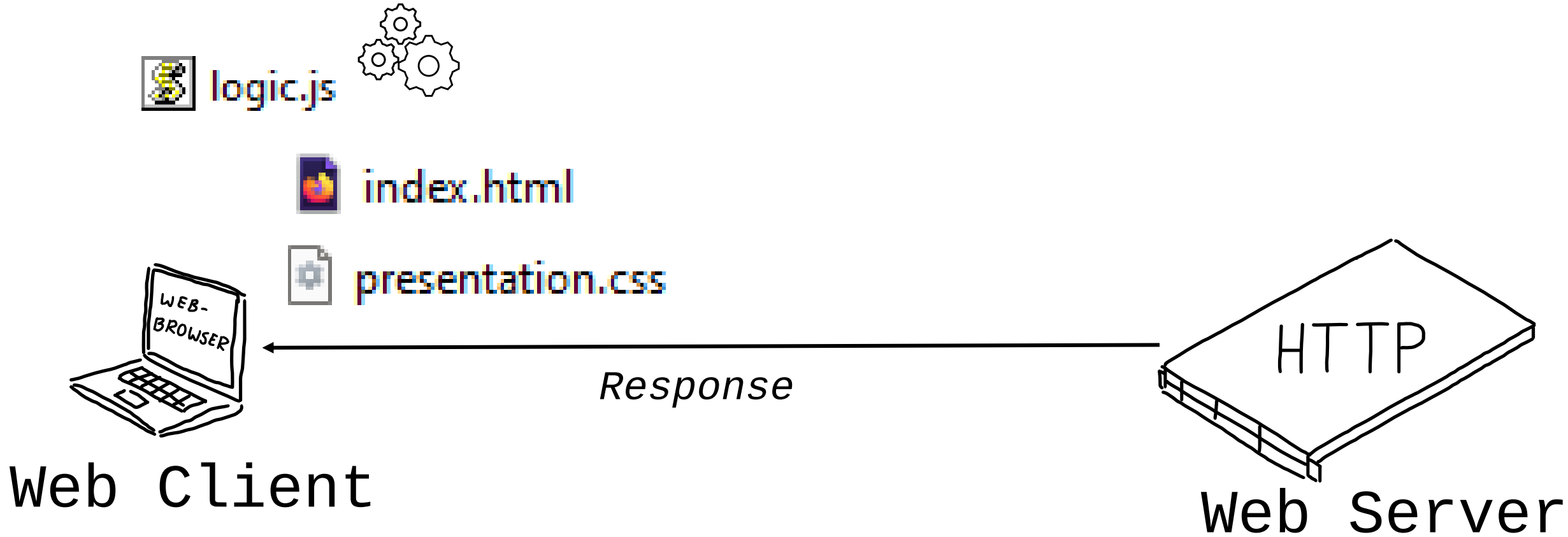
Client-side-scripting



Dynamic Webpage

Dynamisk webbsida

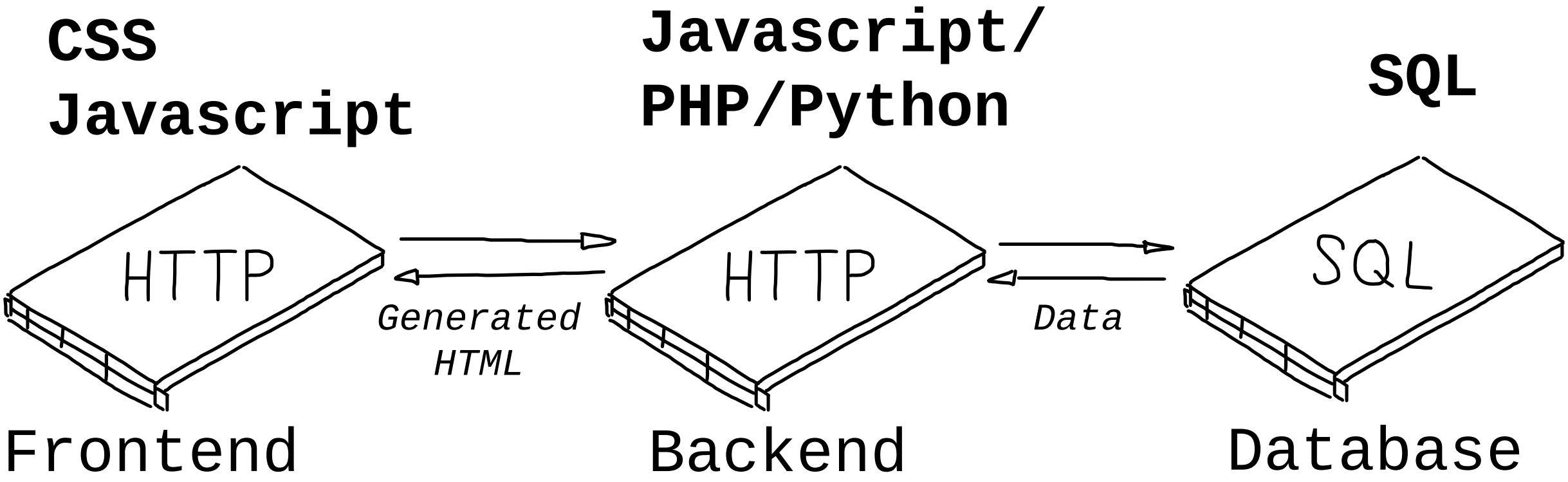
Client-side-scripting



Dynamic Website

Dynamisk webbplats

Sever-side-scripting



LAMP-stack

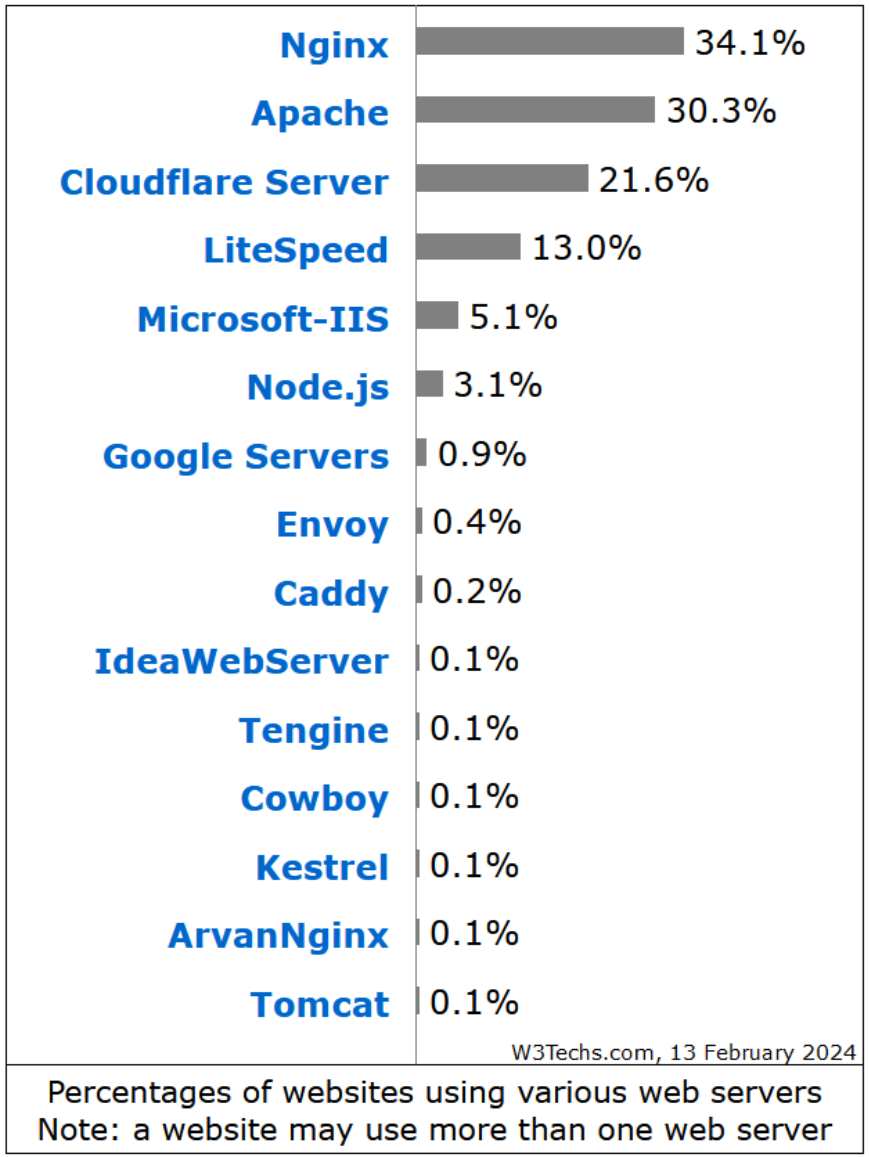
En samling **open source** mjukvaror för **dynamiska webbplatser**

OS	Webbserver	Databas	Server-side-scripting

Web Server

Web Servers

Populäraste webbserver demonerna:



https://w3techs.com/technologies/overview/web_server

Webroot

Den katalog där **webbplatsens** filer finns

```
nics@linux:~$ cat /etc/nginx/nginx.conf
<output omitted>
     root       /usr/share/nginx/html;
<output omitted>
```

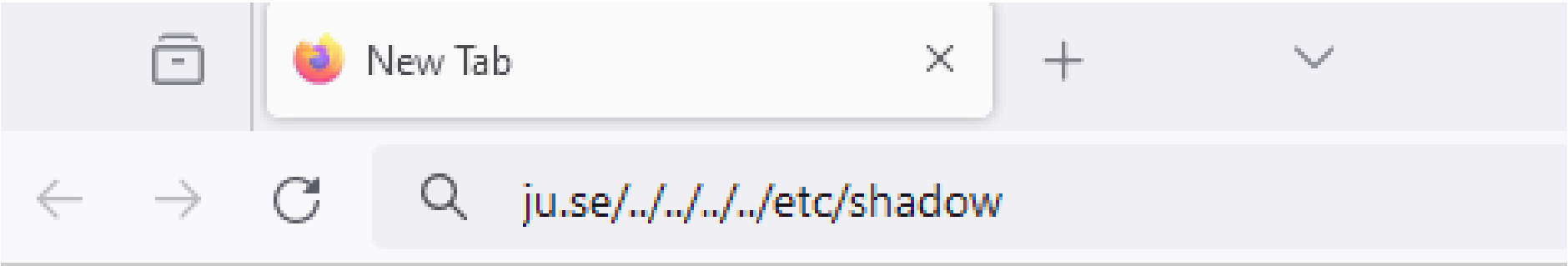
```
nics@linux:~$ ls /usr/share/nginx/html/
404.html  50x.html index.html
```

Path Traversal Attack

Varför används webroot…?

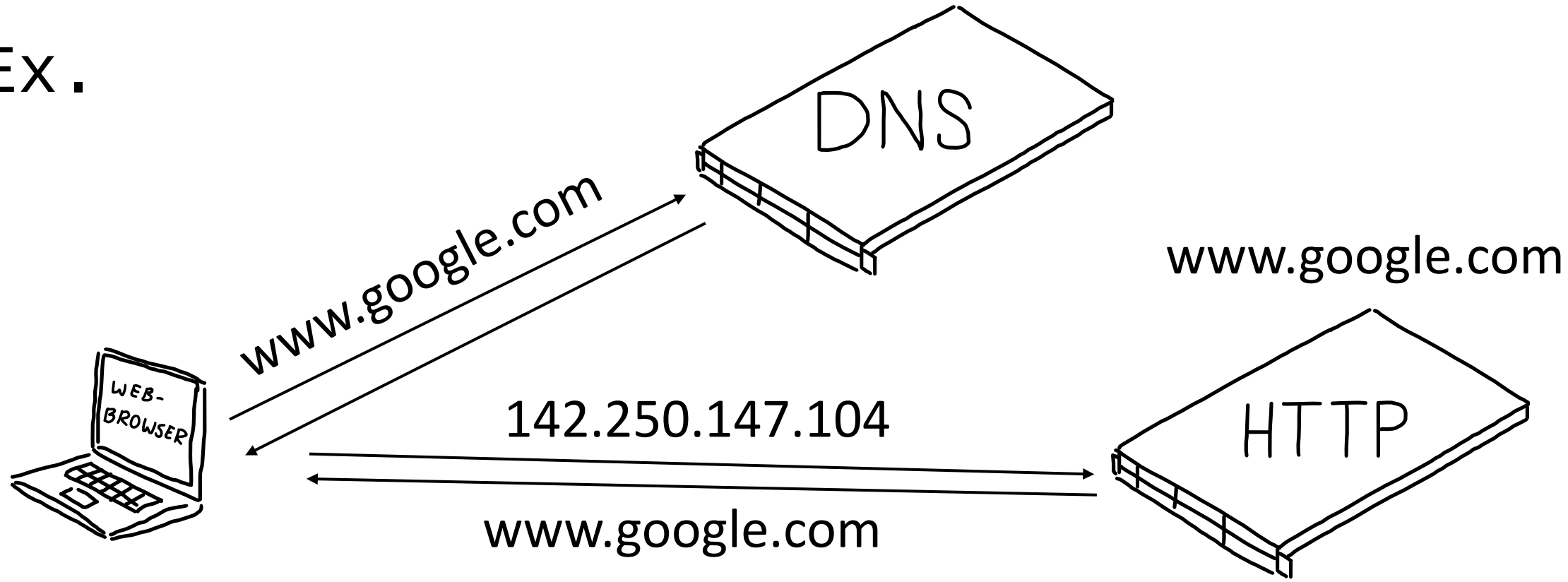
```
nics@linux:~$ cd /usr/share/nginx/html/
nics@linux:/usr/share/nginx/html/$ ls -l ../../../../etc/shadow
---------- root root /etc/shadow
nics@linux:/usr/share/nginx/html/$ sudo ps aux | grep nginx
root       nginx: master process /usr/sbin/nginx
```

…annars kan klienter göra:



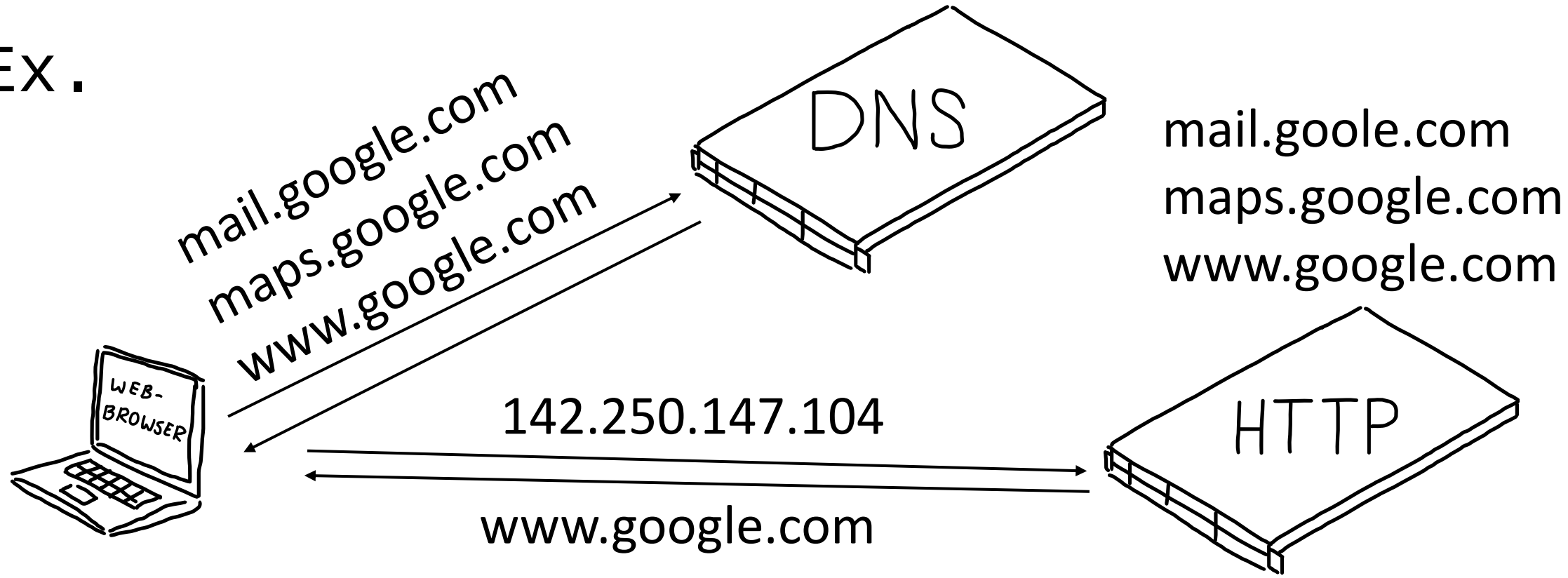
*Hur vet **webbservern** vilken **webbplats** du vill ta dig till?*

Ex.



*Om klienten ansluter till en IP-adress, hur vet server vilken **webbplats** den ska svara med?*

Ex.



Livedemo

Wireshark

Host Header

http

No.	Time	Source	Destination	Protocol	Length	Info
5…	44.414…	192.168.1.69	142.251.9.99	HTTP	132	GET / HTTP/1.1
5…	44.503…	142.251.9.99	192.168.1.69	HTTP	880	HTTP/1.1 200 OK (text/html)

```
GET / HTTP/1.1
Host: www.google.com
User-Agent: curl/7.83.1
Accept: */*
```

```
HTTP/1.1 200 OK
Date: Sun, 05 Feb 2023 13:10:14 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
```

```
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
    [Expert Info (Chat/Sequence): GET / HTTP/1.1\r\n]
    Request Method: GET
    Request URI: /
    Request Version: HTTP/1.1
  Host: www.google.com\r\n
  User-Agent: curl/7.83.1\r\n
  Accept: */*\r\n
  \r\n
  [Full request URI: http://www.google.com/]
  [HTTP request 1/1]
  [Response in frame: 5166]
```

http://www.google.com/

→ 142.251.9.99 HTTP/1.1 GET "/" Host: www.google.com

Virtual Hosts

Alla domännamn

Specifika domännamn

```
nics@linux:~$ cat /etc/nginx/nginx.conf
<output omitted>
server {
        listen          80;
        listen          [::]:80;
        server_name     _;
        root            /usr/share/nginx/html1;
}
server {
        listen          80;
        listen          [::]:80;
        server_name     www.nics.internal nics.internal;
        root            /usr/share/nginx/html2;
}
<output omitted>
```

Tips

Kontrollera syntaxen innan du applicerar

```
nics@linux:~$ sudo nginx -t
```

Kontrollera loggarna

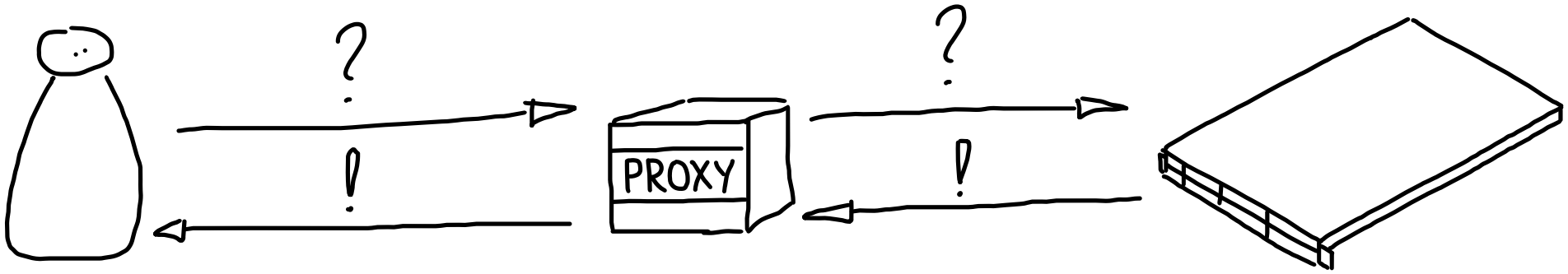
```
nics@linux:~$ sudo journalctl -u nginx
```

```
nics@linux:~$ sudo tail -f
/var/log/nginx/access.log
```

Proxying & Load Balancing

Proxy

Agerar mellanhand åt en klient



Proxy Functions

- Proxying på olika lager
  - IP, TCP, HTTP…
- Vanliga funktioner utöver proxying
  - Caching
  - Filtering/firewalling
  - Malware scanning
  - Monitoring & logging
  - Load balancing
  - TLS termination

Types of Proxy

Delas huvudsakligen upp:

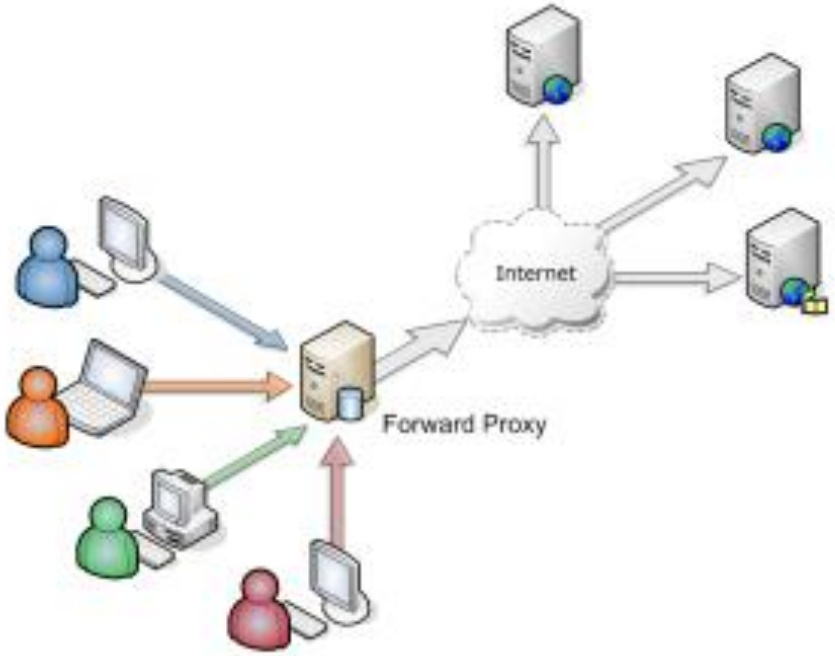
- **Forward Proxy** ‘proxy’
- **Reverse Proxy**

Forward Proxy

Klienter når **webben** genom en **proxy**, och kan t.ex.:

- Cachar svar
- Skydda klienter
  - Filtrering
  - Malware scanning

Blir allt mer ovanligt



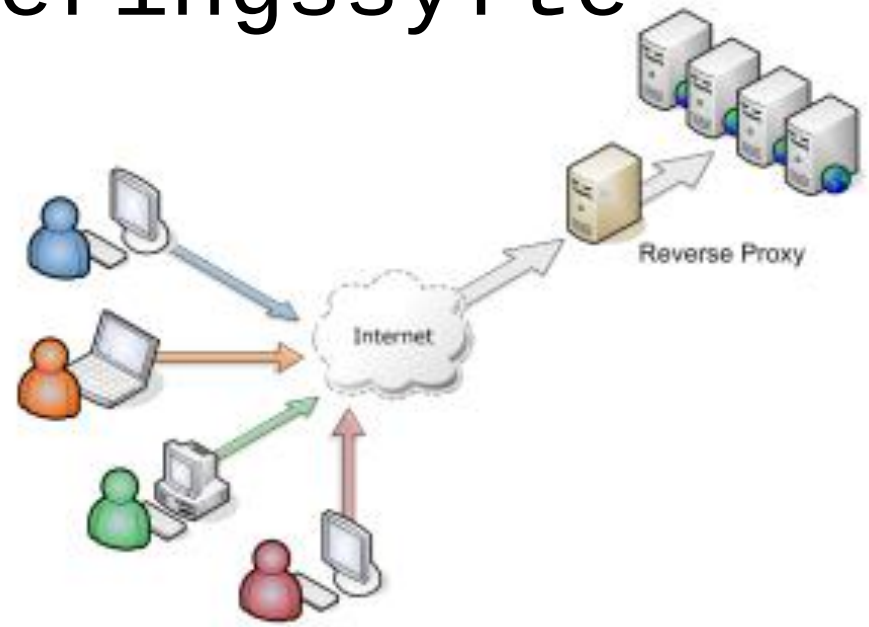
Reverse Proxy

Proxy som placeras framför servrar

Nästan alltid i lastbalanseringssyfte

Vanligt att den även:

- Cachar svar
- TLS-terminerar
  ’offloading’



Load Balancing

Balanserar baserat på olika algoritmer, bland annat:

- **Round Robin**
  Skicka vidare frågor jämnt fördelat i turordning
- **Least Connections**
  Skicka vidare frågor till den med minst antal TCP-sessioner

Load Balancing

Balanserar baserat på olika algoritmer, bland annat:

- **Source**

  Fördelar frågor jämnt baserat på klientens source IP-adress.

  Alla klientens frågor kommer då till samma server

  ’best-effort sticky’

Health Checking

Lastbalansering bör kombineras med **health checks**

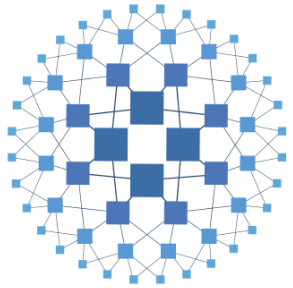
- **Active**
  - Skickar kontrolltrafik
- **Passive**
  - Observerar TCP- eller HTTP-fel i svarstrafiken från servern

Active Health Checking

Health Check Method	Expected Healthy Response
ICMP Echo Request	ICMP Echo Response
TCP SYN	TCP ACK
HTTP OPTIONS/GET/HEAD Request	HTTP Response 200 OK

HAProxy

Mycket populär **reverse proxy**, **load balancer** för TCP- & HTTP-tjänster



HAPROXY

Tips

Kontrollera syntaxen innan du applicerar

```
nics@linux:~$ sudo haproxy -f /etc/haproxy/haproxy.cfg -c
```

Kontrollera loggarna

```
nics@linux:~$ sudo journalctl -u haproxy
nics@linux:~$ sudo journalctl -fu haproxy
```

VRRP

Virtual **R**outer **R**edundancy **P**rotocol

VRRP låter flera enheter dela på en virtuell adress

Öppen version av Cisco **HSRP**

VRRP

Tillstånd	Förklaring
Initilize	Vid uppstart
Master	Använder den virtuella IP-adressen. Skickar ut **VRRP Advertisement** multicast
Backup	Lyssnar efter **VRRP Advertisement**. Vid timeout tar över och blir **Master**

VRRP State

Finns två **states: MASTER** & **BACKUP**

- **MASTER** ska alltid ha högst prioritet
- Default priority är 100

Om **MASTER** slutar skicka **VRRP Advertisment** blir den **BACKUP** med högst **priority MASTER**

Keepalived

VRRP implementeras i Linux med **keepalived**

Vilket även stödjer funktionen: **tracking**

- Tillåter justering av prioritet baserat på om en process körs

```
nics@linux:~$ cat /etc/keepalived/keepalived.conf
vrrp_track_process track_haproxy {
      process haproxy
      weight 50
}
<output omitted>
```

Tips

Kontrollera syntaxen innan du applicerar

```
nics@linux:~$ sudo keepalived -t
```

Kontrollera loggarna

```
nics@linux:~$ sudo journalctl -u keepalived
```

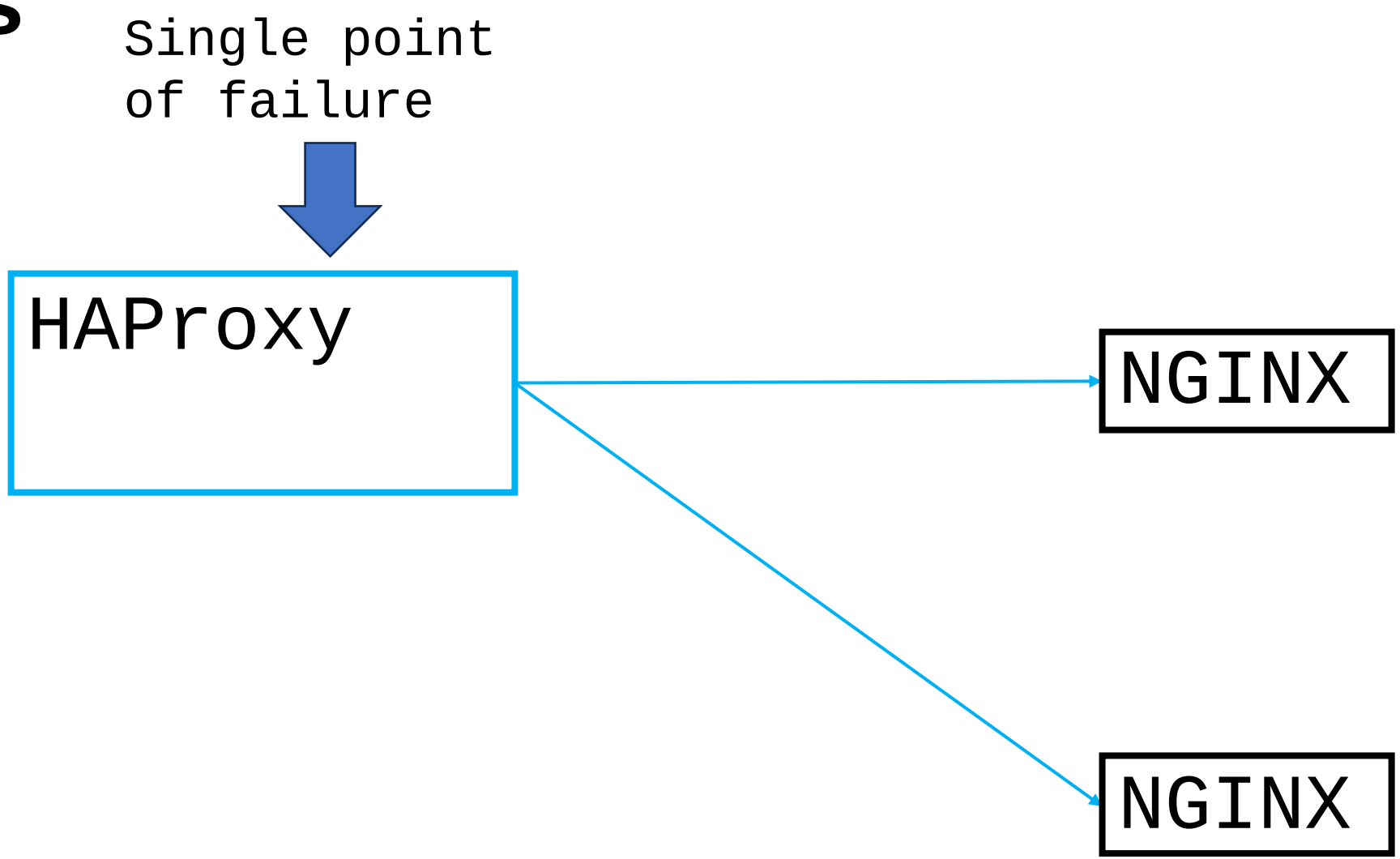
Redundant Arkitektur

Ingen redundans

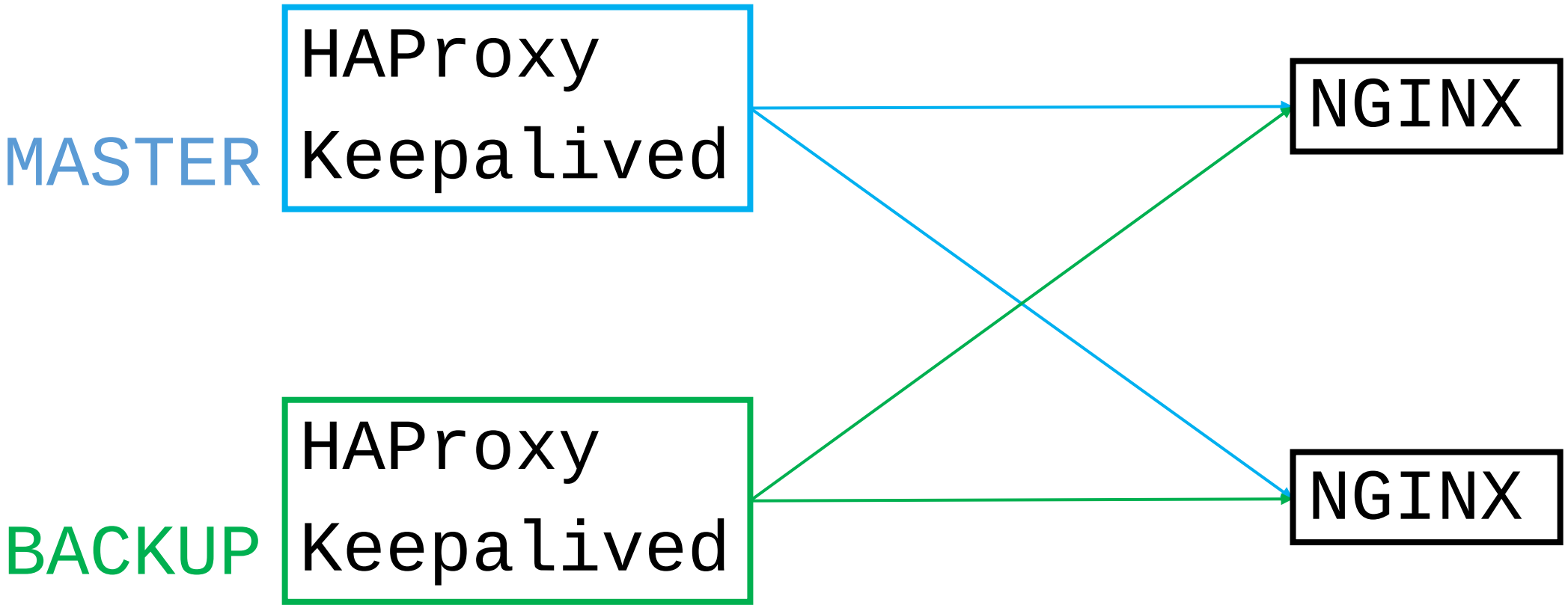
Single point
of failure

NGINX

Redundans



Redundans


MASTER
HAProxy
Keepalived
BACKUP
HAProxy
Keepalived
NGINX
NGINX

Till nästa gång

- Läs kurslitteraturen!

Tack för idag